Jev Router Demo — Build Document

Purpose

Demonstrate that a small, fast routing model ("Jev") can classify incoming CRM queries by complexity and dispatch them to the most cost-effective LLM tier — delivering equivalent answer quality at lower cost, lower latency, and fewer tokens compared to sending every query through a single large model.

The demo targets the existing **DEMO.CRM.CRM_SCM_AGENT** Cortex Agent, which serves ACME Corporation CRM and supply chain data through two tools:

- **Analyst** — text-to-SQL via `DEMO.CRM.CRM_SCM_SEMANTIC_VIEW`
- **ComplianceReview** — rule-based contract compliance evaluation

---

Test Scenarios

Scenario A — Single Model (Baseline)

Every query is processed by **one model** for all orchestration, tool selection, SQL generation, and answer synthesis. This is the control group.

```
User Query → Single Model → Agent Tools → Response
```

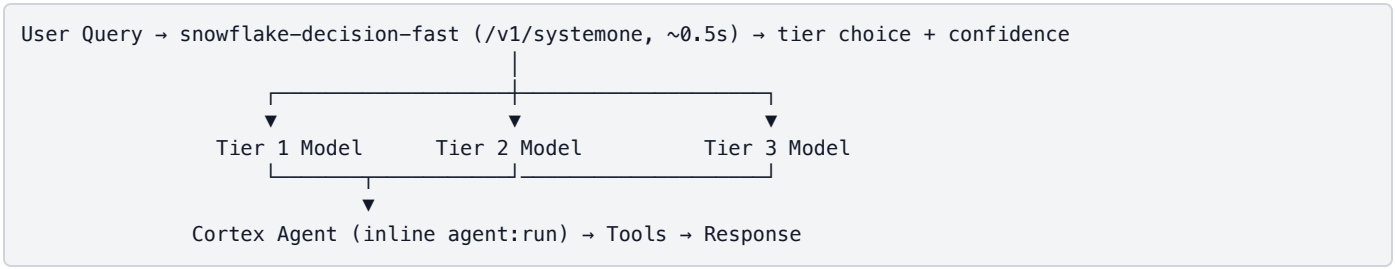
All LLM invocations (orchestration, tool calls, final answer) use the same model. The model is configurable via the `.env` file.

Scenario B — Jev-Routed (Stand-in) — DEFERRED (later stage)

A general-purpose small LLM classifies each query, as a comparison point for the purpose-built Jev router in Scenario C. Not implemented yet. Note: the small Mistral/Llama models originally planned are not served by this account's AI Gateway; use `openai-gpt-5-nano` or `claude-haiku-4-5` as the stand-in classifier.

Scenario C — Jev Router (`snowflake-decision-fast` via AI Gateway) — IMPLEMENTED

The router is TypeSafe's Jev decision model, served on the Snowflake AI Gateway as `snowflake-decision-fast`. It is called with TypeSafe's **System One** API format (https://docs.typesafe.ai/api/system-one) at `POST <gateway>/v1/systemone`, authenticated with the same Snowflake PAT. No external TypeSafe key is required.



How routing works:

- The router asks a single `choice` question (`tier1` / `tier2` / `tier3`, each with descriptive criteria). The response returns the choice, per-tier `probabilities`, and a `confidence` score.
- **Low-confidence escalation:** if confidence < `ROUTER_MIN_CONFIDENCE` (default 0.25), the query moves up one tier, so router uncertainty costs money rather than quality.
- **Cost:** router input tokens only (TypeSafe documents output tokens as free; observed ~190 input tokens per call).

Verified findings (2026-10-20, Demo account):

Finding	Detail
Endpoint	`/v1/systemone` works; `/v1/chat/completions` returns HTTP 400 "model does not support this endpoint"
Router latency	~0.4–0.9s per call
Router accuracy (30-question bank)	93% raw (28/30); errors: t1-10 over-routed to T2, t3-06 under-routed to T2 (confidence 0.14, so escalation catches it)
Escalation threshold	0.6 over-escalated 8 correctly-routed queries → default lowered to 0.25

> The direct TypeSafe endpoint (`https://api.typesafe.ai/v1/systemone`, model `jev-latest`) also works with the same client by changing `TYPESAFE_JEV_API_URL`, `TYPESAFE_JEV_MODEL`, and injecting `TYPESAFE_JEV_API_KEY`.

Implementation notes (Scenarios A and C)

- **Model override requires an inline agent run.** The named-agent endpoint (`/api/v2/databases/{db}/schemas/{s}/agents/{name}:run`) cannot override `models`. The harness fetches the agent's spec once (instructions, orchestration budget, tools, tool_resources) via REST, then calls `POST /api/v2/cortex/agent:run` with `models.orchestration` set per scenario/tier, so both scenarios run an identical agent.
- **AI Gateway** serves the router (`/v1/systemone`) and the judge (`/v1/messages` for Claude, `/v1/chat/completions` otherwise). Agent orchestration runs on the Cortex Agent API, whose token usage is reported per model in the final `response` event.
- **Gateway model availability (this account):** Claude and OpenAI models work. `mistral-large3` and `ministral-3-8b` return "not available for this account"; `llama3.1-8b`, `llama3.3-70b`, and `gemma-4-e2b` return "unknown model".
- **Agent orchestration models verified:** `claude-haiku-4-5` (~11s for a T1 query), `openai-gpt-5-mini` (~19s), `claude-sonnet-4-5`.
- **Hostnames:** use the hyphenated account host (`sfsenorthamerica-jgarcia-demo-uswest2`); Python TLS rejects the underscore form that `DESCRIBE AI GATEWAY` returns.
- **Pricing:** credit rates per 1M tokens were derived from this account's actual billing (`ACCOUNT_USAGE.CORTEX_AGENT_USAGE_HISTORY` / `AI_GATEWAY_USAGE_HISTORY`) and stored in `config/pricing.yaml`. Each run also writes `reconcile.sql` to pull billed credits for its time window.
- **Question bank:** contract numbers in the data are `ACME-10000`–`ACME-10020`; the Tier 3 compliance questions use them.

Question Tiers

Tier 1 — Simple Lookups & Aggregations

Single-table queries, direct counts, basic filters. Any small model produces the same SQL and answer as a large model.

#	Example Question
1	How many active contracts do we have?
2	List all suppliers with Critical risk level
3	What is the total number of customers?
4	Show me all products in the Routing category
5	How many shipments are currently in transit?
6	What currencies do we support?
7	List all countries in the EMEA region
8	How many open compliance exceptions exist?
9	What is the average contract line value?
10	Show the most recent 5 contracts by start date

Expected traffic share: 60–70%

Tier 2 — Multi-Step Analytical Questions

Multi-filter, join-heavy, or comparative queries requiring decomposition. A mid-tier model handles these well.

#	Example Question
1	Which customers in EMEA have contracts expiring in the next 90 days with total line value over $1M?
2	Compare supplier risk distribution across regions
3	Top 10 products by total contract line revenue with their primary supplier and risk level
4	What is the month-over-month trend in new contract signings this year?
5	Which suppliers have both High risk and shipments currently delayed?
6	Break down contract value by customer country and product type
7	List customers who have contracts with more than 3 different product types
8	What percentage of shipment lines were delivered late by supplier?
9	Show contracts where the total line value exceeds the contract header value by more than 10%
10	Rank regions by total active contract value and count of unique customers

Expected traffic share: 20–25%

Tier 3 — Complex Reasoning & Compliance

Compliance reviews, multi-tool orchestration, synthesis across domains, or questions requiring judgment.

#	Example Question
1	Run a compliance review on contract ACME-10001
2	Review compliance for contracts ACME-10000 through ACME-10020
3	Which contracts are at risk due to both supplier risk AND compliance violations?
4	Summarize our exposure to sanctioned countries across all active contracts and recommend remediation
5	For each compliance exception that is still open, identify the root cause and suggest a resolution path
6	Compare the compliance posture of our top 5 customers by contract value
7	Identify contracts that use restricted products shipped through high-risk suppliers
8	Build a risk matrix of contracts by combining supplier risk, country risk, and compliance status
9	What is the financial exposure from contracts that would fail a compliance review?
10	Generate an executive summary of our compliance health with specific action items

Expected traffic share: 5–15%

Model Configuration

All model assignments are defined in a `.env` file at the project root. Every model in every scenario is independently configurable — swap any model without code changes.

Model Availability (verified on the Demo account's AI Gateway / Agent API)

Category	Works	Notes
Router	`snowflake-decision-fast`	Jev decision model; `/v1/systemone` endpoint only
Small / fast	`claude-haiku-4-5`, `openai-gpt-5-nano`, `openai-gpt-5-mini`	Tier 1–2 candidates
Large / capable	`claude-sonnet-4-5`, `claude-sonnet-4-6`, `claude-opus-4-8`, `openai-gpt-5.4`	Tier 3 / baseline / judge
Not served on this gateway	`mistral-large3`, `ministral-3-8b` (not available for account); `llama3.1-8b`, `llama3.3-70b`, `gemma-4-e2b` (unknown model)	Earlier drafts recommended some of these

.env Configuration (actual file in the project root)

The Snowflake PAT is **not** stored in `.env`; it is injected at runtime as `SNOWFLAKE_PAT` from the stored secret:

```
SNOWFLAKE_PAT="<snowflake_ai_pat>" uv run jev-demo --scenario all
```

```
# ================================================
# Snowflake connection
# ================================================
# Account host (no https://). Use hyphens, not underscores — Python TLS rejects underscore hosts.
SNOWFLAKE_ACCOUNT_HOST=sfsenorthamerica-jgarcia-demo-uswest2.snowflakecomputing.com
# AI Gateway base URL (from DESCRIBE AI GATEWAY SNOWFLAKE -> base_url)
AI_GATEWAY_BASE_URL=https://sfsenorthamerica-jgarcia-demo-
uswest2.snowflakecomputing.com/api/v2/aigateways/snowflake
# The PAT is NOT stored here. Inject it at runtime as SNOWFLAKE_PAT, e.g.:
#   SNOWFLAKE_PAT="<snowflake_ai_pat>" uv run jev-demo --scenario all
# Agent whose tools/instructions are replicated for every run
SNOWFLAKE_AGENT_FQN=DEMO.CRM.CRM_SCM_AGENT
# Warehouse used by agent tools
AGENT_WAREHOUSE=COMPUTE_WH

# ================================================
# Scenario A — Single Model Baseline
# ================================================
# All orchestration, tool selection, and synthesis use this one model.
# Recommended: claude-sonnet-4-5 (high quality, strong baseline)
# Alternatives: claude-sonnet-4-6, claude-opus-4-8, openai-gpt-5.4
BASELINE_MODEL=claude-sonnet-4-5

# ================================================
# Scenario C — Jev Router (snowflake-decision-fast via AI Gateway)
# ================================================
# Router endpoint: TypeSafe System One API format (https://docs.typesafe.ai/api/system-one).
# Recommended: the Snowflake AI Gateway /v1/systemone path (uses the same Snowflake PAT).
# Alternative: https://api.typesafe.ai/v1/systemone (requires TYPESAFE_JEV_API_KEY below)
TYPESAFE_JEV_API_URL=https://sfsenorthamerica-jgarcia-demo-
uswest2.snowflakecomputing.com/api/v2/aigateways/snowflake/v1/systemone
# Router model.
# Recommended: snowflake-decision-fast (Jev-class decision model on the AI Gateway)
# Alternatives: snowflake-decision; jev-latest (TypeSafe direct endpoint only)
TYPESAFE_JEV_MODEL=snowflake-decision-fast
# Only needed when calling TypeSafe directly. Leave empty to use SNOWFLAKE_PAT.
# Store it with /secrets and inject at runtime as TYPESAFE_JEV_API_KEY — never paste it here.
TYPESAFE_JEV_API_KEY=
# Minimum router confidence; below it the query escalates one tier (safety net against under-routing).
# Recommended: 0.25 (measured: raw routing 93% accurate; 0.6 over-escalated 8/30 correctly routed queries)
# Set to 0 to disable escalation.
ROUTER_MIN_CONFIDENCE=0.25

# Tier 1 — Simple lookups and aggregations.
# Recommended: claude-haiku-4-5 (fast, cheap, verified as agent orchestrator)
# Alternatives: openai-gpt-5-mini, openai-gpt-5-nano
TYPESAFE_TIER1_MODEL=claude-haiku-4-5

# Tier 2 — Multi-step analytical questions.
# Recommended: claude-haiku-4-5 (handles multi-join analytics well at low cost)
# Alternatives: openai-gpt-5-mini, claude-sonnet-4-5
TYPESAFE_TIER2_MODEL=claude-haiku-4-5

# Tier 3 — Complex reasoning and compliance reviews.
# Recommended: claude-sonnet-4-5 (same as baseline, so Tier 3 quality is unchanged)
# Alternatives: claude-sonnet-4-6, claude-opus-4-8, openai-gpt-5.4
TYPESAFE_TIER3_MODEL=claude-sonnet-4-5

# ================================================
# Evaluation
# ================================================
# LLM-as-judge model for scoring answer quality (called via AI Gateway /v1/messages).
# Recommended: claude-sonnet-4-5 (keep consistent across all runs for stable scoring)
# Alternatives: claude-opus-4-8 (stricter, more expensive evals)
JUDGE_MODEL=claude-sonnet-4-5
```

```
# Set to false to skip quality scoring (faster, cheaper smoke runs)
ENABLE_JUDGE=true

# ================================================
# Run settings
# ================================================
# Pricing table (credits per 1M tokens) used for cost estimates
PRICING_FILE=config/pricing.yaml
# USD per Snowflake credit (contract-specific)
CREDIT_PRICE_USD=3.00
# Per-request timeout in seconds
REQUEST_TIMEOUT_S=300
# Override the agent object's orchestration budget for BOTH scenarios (leave empty to keep the agent's own).
# The CRM_SCM_AGENT spec caps runs at 60s / 32000 tokens, which truncates many Tier 2-3 answers
# ("reached time limit"). Recommended for this benchmark: 180 seconds, 64000 tokens.
AGENT_BUDGET_SECONDS=180
AGENT_BUDGET_TOKENS=64000
```

Recommended Model Candidates

Role	Default	Alternatives
Baseline (Scenario A)	`claude-sonnet-4-5`	`claude-sonnet-4-6`, `claude-opus-4-8`, `openai-gpt-5.4`
Router (Scenario C)	`snowflake-decision-fast`	`snowflake-decision`, `jev-latest` (TypeSafe direct)
Tier 1	`claude-haiku-4-5`	`openai-gpt-5-mini`, `openai-gpt-5-nano`
Tier 2	`claude-haiku-4-5`	`openai-gpt-5-mini`, `claude-sonnet-4-5`
Tier 3	`claude-sonnet-4-5`	`claude-sonnet-4-6`, `claude-opus-4-8`, `openai-gpt-5.4`
Judge	`claude-sonnet-4-5`	`claude-opus-4-8` (keep constant across runs)

Measurements

Every query execution captures granular metrics at each LLM invocation step. The metrics are collected separately for Scenario A, Scenario B, and Scenario C.

Per-Query Metrics

1. Performance

Metric	Description	Captured At
`total_latency_ms`	Wall-clock time from query submission to final answer	End-to-end
`router_latency_ms`	Time for the router to classify the query (Scenario B: AI Gateway; Scenario C: TypeSafe API)	Router step
`orchestration_latency_ms`	Time for the agent to select tools and plan execution	Orchestration step
`tool_execution_latency_ms`	Time for each tool call (SQL execution, compliance review)	Per tool call
`synthesis_latency_ms`	Time for the model to compose the final answer	Final answer step
`time_to_first_token_ms`	Time from query submission to first streamed token (if applicable)	End-to-end

2. Cost

Metric	Description	Captured At
`router_input_tokens`	Tokens sent to the router model (Scenario B); N/A for Scenario C	Router step (Scenario B)
`router_output_tokens`	Tokens produced by the router model (Scenario B); N/A for Scenario C	Router step (Scenario B)
`router_api_cost`	Cost of the TypeSafe Jev API call (Scenario C only)	Router step (Scenario C)
`orchestration_input_tokens`	Tokens sent for orchestration/tool planning	Orchestration step
`orchestration_output_tokens`	Tokens produced for orchestration/tool planning	Orchestration step
`synthesis_input_tokens`	Tokens sent for final answer generation	Synthesis step
`synthesis_output_tokens`	Tokens produced for final answer generation	Synthesis step
`total_input_tokens`	Sum of all input tokens across all LLM calls	Aggregated
`total_output_tokens`	Sum of all output tokens across all LLM calls	Aggregated
`total_tokens`	`total_input_tokens` + `total_output_tokens`	Aggregated
`estimated_cost_usd`	Estimated cost based on per-model token pricing	Aggregated

> Token counts are captured per LLM invocation. Cost is derived from published per-model pricing applied to each invocation's model.

3. Quality

Metric	Description	Method
`answer_correctness`	Does the answer match the expected result? (1–10)	LLM-as-judge
`answer_completeness`	Does the answer address all parts of the question? (1–10)	LLM-as-judge
`answer_relevance`	Is the answer on-topic and free of hallucination? (1–10)	LLM-as-judge
`quality_score`	Weighted average of correctness, completeness, relevance	Computed
`sql_valid`	Did the generated SQL execute without error? (boolean)	Runtime check
`tool_selection_correct`	Was the right tool selected (Analyst vs ComplianceReview)? (boolean)	Rule-based check

4. Token Breakdown by Step

Each query produces a structured record with token counts per step:

```
{
  "query_id": "q-001",
  "scenario": "jev-routed",
  "tier": 1,
  "steps": [
    {
      "step": "router",
      "model": "mistral-large2",
      "input_tokens": 320,
      "output_tokens": 15,
      "latency_ms": 180
    },
    {
      "step": "orchestration",
      "model": "mistral-large2",
      "input_tokens": 1200,
      "output_tokens": 85,
      "latency_ms": 420
    },
    {
      "step": "tool_execution",
      "tool": "analyst",
      "latency_ms": 310
    },
    {
      "step": "synthesis",
      "model": "mistral-large2",
      "input_tokens": 800,
      "output_tokens": 150,
      "latency_ms": 350
    }
  ],
  "totals": {
    "total_input_tokens": 2320,
    "total_output_tokens": 250,
    "total_tokens": 2570,
    "total_latency_ms": 1260,
    "estimated_cost_usd": 0.0038
  }
}
```


Aggregate Metrics (Per Scenario)

Metric	Description
`avg_latency_ms`	Mean total latency across all queries
`p50_latency_ms`	Median total latency
`p95_latency_ms`	95th percentile total latency
`avg_tokens_per_query`	Mean total tokens consumed per query
`total_cost_usd`	Sum of estimated cost across all queries
`avg_cost_per_query_usd`	Mean cost per query
`avg_quality_score`	Mean quality score across all queries
`quality_score_by_tier`	Mean quality score broken down by tier
`latency_by_tier`	Mean latency broken down by tier
`cost_by_tier`	Mean cost broken down by tier

Scenario Comparison (Final Report)

Metric	Scenario A (Single Model)	Scenario B (Stand-in Router)	Scenario C (TypeSafe Jev)	B vs A Delta	C vs A Delta	C vs B Delta
Avg Latency	—	—	—	—	—	—
P95 Latency	—	—	—	—	—	—
Avg Cost/Query	—	—	—	—	—	—
Total Cost	—	—	—	—	—	—
Avg Tokens/Query	—	—	—	—	—	—
Router Latency	N/A	—	—	—	—	—
Router Cost	N/A	—	—	—	—	—
Avg Quality Score	—	—	—	—	—	—
Tier 1 Quality	—	—	—	—	—	—
Tier 2 Quality	—	—	—	—	—	—
Tier 3 Quality	—	—	—	—	—	—
Router Accuracy	N/A	—	—	—	—	—

Technology Stack

Component	Technology
Language	Python 3.12
Project manager	`uv`
Model invocation	Snowflake AI Gateway (Cortex LLM functions via REST or SDK)
Agent interaction	Snowflake Cortex Agent API (`/api/v2/cortex/agent:run`)
External routing	TypeSafe Jev API (Scenario C)
Configuration	`.env` file loaded via `python-dotenv`
Results storage	Local JSON + optional Snowflake table
Reporting	Console summary + HTML scorecard

Key Python Packages

Package	Purpose
`python-dotenv`	`.env` configuration loading
`httpx`	HTTP + SSE streaming for AI Gateway and Agent API calls
`pyyaml`	Question bank and pricing table
`pydantic`	Data validation and metrics schema
`rich`	Console output formatting

Package	Purpose
`jinja2`	HTML report templating

Build Plan

Phase 1 — Single Model Baseline (Scenario A) — IMPLEMENTED

Build and validate the test harness using a single model for all queries. This phase is fully buildable now.

1. **Project scaffolding** — `uv` project, `.env`, directory structure
2. **Question bank** — YAML file with 30 questions (10 per tier), each tagged with tier and expected tool
3. **Agent client** — Python module to call `DEMO.CRM.CRM_SCM_AGENT` via the Cortex Agent API, capturing per-step token counts and latency
4. **Metrics collector** — Pydantic models for per-query and aggregate metrics; writes JSON results
5. **LLM judge** — Evaluator module that scores each answer on correctness, completeness, relevance using the judge model via AI Gateway
6. **Test runner** — Orchestrator that iterates through the question bank, runs each query, collects metrics, runs judge
7. **Reporter** — Console summary via `rich` + HTML scorecard via `jinja2`
8. **Validation** — Run full suite, verify metrics are captured correctly, spot-check judge scores

Phase 2 — Jev-Routed Stand-in (Scenario B) — DEFERRED

Add the routing layer on top of the Phase 1 harness. Buildable now with a stand-in router model; swap to Jev when available.

1. **Router module** — Calls the router model via AI Gateway with a classification prompt; returns tier (1/2/3) and confidence score
2. **Router prompt engineering** — Design and test the classification prompt against the question bank; measure router accuracy
3. **Tier dispatcher** — Maps classified tier to the configured model; passes query + model to the agent client
4. **Extended metrics** — Add router step metrics (tokens, latency, classified tier, confidence) to the per-query record
5. **Comparison runner** — Orchestrator that runs both Scenario A and B on the same question bank
6. **Comparative reporter** — Side-by-side comparison table in console and HTML
7. **Router accuracy report** — Measure how often the router classifies queries into the expected tier
8. **Validation** — Run full comparison suite, verify deltas, sanity-check that quality is equivalent

Phase 3 — Jev Router via `snowflake-decision-fast` (Scenario C) — IMPLEMENTED

Integrate TypeSafe's production Jev routing API as a drop-in replacement for the stand-in router. Requires API access from TypeSafe.

1. **TypeSafe client module** — `typesafe_client.py` — HTTP client for the Jev routing API; sends the user query, receives routing decision (tier, recommended model, confidence, reasoning)
2. **Response adapter** — Normalize the TypeSafe API response into the same internal routing decision schema used by the stand-in router, so the dispatcher and metrics pipeline work unchanged
3. **Model mapping** — Map TypeSafe-recommended model names to Snowflake AI Gateway model identifiers; fall back to tier defaults when a recommended model is not available in the gateway

4. **Extended metrics for Scenario C** — Capture TypeSafe-specific fields: API response time, Jev confidence score, Jev reasoning text, recommended model vs. actual model used, API cost per call
5. **Secret management** — Store the TypeSafe API key via `cortex secret store`; inject at runtime using `python-dotenv` with the `<stored-secret>` pattern
6. **Three-scenario runner** — Extend the comparison orchestrator to run Scenario A, B, and C on the same question bank in a single invocation
7. **Three-way comparative reporter** — Side-by-side-by-side comparison in console and HTML; include B-vs-A, C-vs-A, and C-vs-B deltas
8. **Router accuracy comparison** — Compare stand-in router (B) accuracy vs. TypeSafe Jev (C) accuracy against the expected tier labels
9. **Validation** — Run full three-scenario suite; verify that Scenario C metrics are captured correctly; confirm API key is never logged or written to results

---

Results — Run 20261020-104241 (30 questions × 2 scenarios, 180s agent budget)

Metric	A: `claude-sonnet-4-5` for all	C: Jev-routed	C vs A
Avg cost / query (est.)	$0.2915	$0.1911	**–34.5%**
Avg latency	51.7s	40.5s	**–21.6%**
P95 latency	117.0s	104.4s	–10.7%
Avg time to first token	41.1s	26.4s	–35.8%
Avg tokens / query	168,418	169,900	+0.9%
Avg quality (judge, 1–10)	8.57	8.33	–2.7%
Tool selection correct	100%	100%	—
Errors	0	0	—

Tier	Latency A → C	Cost A → C	Quality A → C
T1	20.7s → 16.4s	$0.1648 → $0.0673 (–59%)	9.24 → 8.51
T2	71.6s → 33.4s	$0.3851 → $0.1329 (–65%)	8.01 → 8.72
T3	62.9s → 71.8s	$0.3246 → $0.3730 (+15%)	8.45 → 7.77

Router (`snowflake-decision-fast`): 93% raw accuracy, ~0.5s and ~194 input tokens per call, 0 under-routed, 2 over-routed.

Observations: - The savings come from Tiers 1–2, where a small model is enough. Routing costs about 0.5s and almost nothing per query. - Token counts barely change: the agent's large semantic-model context dominates and most of it is served from cache. The savings come from cheaper per-token rates, not from using fewer tokens. - Tier 3 runs the same model in both scenarios, so its deltas are run-to-run variance. Quality differences of ±0.7 at this sample size (10 per tier) are inside the noise; repeat runs to tighten the comparison. - Run 20261020-102504 used the agent's own 60s budget. Many Tier 2–3 answers were cut off ("reached time limit") in both scenarios (avg quality 6.21 A vs 7.26 C). The benchmark now sets `AGENT_BUDGET_SECONDS=180`. - Costs are estimates from `config/pricing.yaml`. Run `results/<run>/reconcile.sql` after ACCOUNT_USAGE catches up to get billed credits. Judge cost ($1.95 for this run) and warehouse SQL compute are excluded.

Running

```
# Router accuracy only (fast, ~15s)
SNOWFLAKE_PAT="<snowflake_ai_pat>" uv run jev-demo --route-only
# Smoke test: one question per tier, both scenarios
SNOWFLAKE_PAT="<snowflake_ai_pat>" uv run jev-demo --ids t1-01,t2-04,t3-01
# Full benchmark (30 questions x 2 scenarios)
SNOWFLAKE_PAT="<snowflake_ai_pat>" uv run jev-demo --scenario all --concurrency 3
```

Options: `--scenario a|c|all`, `--tier N` (repeatable), `--ids`, `--limit N` (per tier), `--concurrency N`, `--no-judge`.

Directory Structure

```
CRM-Jev/
├── build-jev.md               # This document
├── .env                       # Models, endpoints, run settings (no secrets)
├── pyproject.toml             # uv project (Python 3.12.15); entry point: jev-demo
├── config/pricing.yaml        # Credits per 1M tokens, derived from account billing
├── questions/question_bank.yaml  # 30 questions, tier + expected tool
├── sql/derive_pricing.sql     # Re-derive pricing.yaml rates from ACCOUNT_USAGE
├── src/jev_demo/
│   ├── config.py              # .env + runtime secret loading
│   ├── agent_client.py        # Inline Cortex agent:run, SSE streaming, per-step timing
│   ├── router.py              # Jev router (snowflake-decision-fast, System One API)
│   ├── judge.py               # LLM-as-judge via AI Gateway
│   ├── pricing.py             # Token -> credits -> USD
│   ├── metrics.py             # Pydantic schemas
│   ├── runner.py              # CLI orchestrator
│   └── reporter.py            # Aggregates, console, HTML, reconcile.sql
├── templates/scorecard.html   # Comparison report template
└── results/<run-id>/          # scenario_a.json, scenario_c.json, summary.json, comparison.html, reconcile.sql
```